1. What is React?

React is a **JavaScript library** used to build **user interfaces (UI)**, especially **single-page applications (SPA)**.

- Developed by **Facebook (Meta)**
- Used to create **fast, interactive, and reusable UI components**
- Focuses only on the **View layer** (UI)

  React is not a framework. It is a **library**.

---

2. Why React Was Created (Problem Before React)

Traditional Website Flow (Before React)

```
User clicks button
      ↓
Browser sends request to server
      ↓
Server creates full HTML page
      ↓
Browser reloads entire page
```

Problems:

- Full page reload ❌
- Slow user experience ❌
- Too much server dependency ❌
- Difficult to manage UI state ❌

---

3. React Solution (Modern Way)

React Website Flow

```
User clicks button
      ↓
React updates only required part of UI
      ↓
No page reload
```

Benefits:

- Faster UI ✅
- Smooth experience ✅
- Less server load ✅
- Better code structure ✅

---

4. What is SPA (Single Page Application)?

In React:

- Only **one HTML page** is loaded (`index.html`)
- Content changes dynamically using JavaScript

```
index.html (loaded once)
      ↓
React controls everything
```

Examples:

- Gmail
- Facebook
- Instagram

---

5. Why Use React?

Key Reasons

1. **Component-Based Architecture**
2. **Reusable Code**
3. **Virtual DOM for Performance**
4. **Strong Community Support**
5. **Easy Integration with Backend (Django, Node, etc.)**

---

6. Component-Based Architecture (Very Important)

Without Components (Messy)

```
HTML + JS + CSS mixed
Hard to maintain
```

With Components (React)

```
App
 ├── Navbar
 ├── Sidebar
 ├── Content
 │     ├── Card
 │     └── Button
 └── Footer
```

Each component:

- Has its own logic
- Can be reused
- Easy to debug

---

7. Virtual DOM (Core Concept)

Real DOM (Slow)

```
Change UI
   ↓
Browser updates full DOM
```

Virtual DOM (Fast)

```
Change State
    ↓
Virtual DOM compares changes (Diffing)
    ↓
Updates only changed elements
```

Result:

- Faster rendering
- Better performance

---

8. React vs Traditional JavaScript

Feature	JavaScript	React
Page Reload	Yes	No
UI Update	Manual	Automatic
Code Reuse	Difficult	Easy
Performance	Slow	Fast

---

9. How React Works (Diagram)

```
User Action
     ↓
State Change
     ↓
Virtual DOM
     ↓
Compare with Real DOM
     ↓
Update only changed part
```

---

10. Installing React (Step-by-Step)

Step 1: Install Node.js

- React needs **Node.js**
- Node provides:
    - `npm` (Node Package Manager)

Check installation:

```
node -v
npm -v
```

Step 2: Create React App (Modern Way)

```
npx create-react-app myapp
```

What this does:

- Downloads React
- Sets up folder structure
- Configures Webpack & Babel

Step 3: Run React App

```
cd myapp
npm start
```

- Runs on: `http://localhost:3000`
- Hot reload enabled

---

11. React Folder Structure (Basic)

```
myapp
 ├── node_modules
 ├── public
 │    └── index.html
 ├── src
 │    ├── App.js
 │    ├── index.js
 │    └── components
 └── package.json
```

Important Files

- `index.html` → Single HTML file
- `index.js` → Entry point
- `App.js` → Main component

---

12. React with Backend (Django Example)

```
React (Frontend)
   ↓ API calls
Django (Backend)
   ↓
Database
```

React handles:

- UI
- Routing

Django handles:

- Authentication

- Database
- Business logic

---

13. When to Use React?

Use React if:

- UI is complex
- Data changes frequently
- Performance matters

Do NOT use React if:

- Simple static website
- No user interaction

---

14. First Lecture Summary

- React is a UI library
- Uses component-based architecture
- Uses Virtual DOM
- Creates Single Page Applications
- Installed using Node & npm

Interview One-Liners ⭐

- React is a JavaScript library for building UI
- React uses Virtual DOM
- React creates Single Page Applications
- Components make code reusable